

Project Proposal

Database Management Systems Laboratory

CS39202

Be Positive with Lurking Crabs

B⁺ Tree over a Custom Buffer Pool Manager with LRU-K Eviction and Latch Crabbing for Concurrency

Group BSDM

Name	Roll No
Shrey Patel	23CS10051
Arnav Singh	23CS30009
Arham Bhansali	23CS30007

1. Abstract

Modern database management systems rely on two fundamental components working in tandem: a buffer pool manager that mediates all access between in-memory pages and on-disk storage, and a tree-based indexing structure that enables efficient data retrieval. This project implements both components from scratch in C++ and couples them together to study their interaction under realistic workloads.

The buffer pool manager maintains a fixed-size pool of frames in RAM. Every page required by the index is fetched through this manager. When the pool is full and a new page is needed, the manager evicts a victim page using the LRU-K eviction policy, a generalization of LRU that tracks the K -th most recent access to make smarter eviction decisions under repeated access patterns.

On top of the buffer pool, we implement a B⁺ Tree index where each tree node corresponds to exactly one buffer pool page. The tree supports insertion, deletion, point search, and range scan. To enable safe concurrent access by multiple threads, we implement latch crabbing (also called lock coupling): a thread acquires a latch on a child page before releasing the parent's latch, allowing concurrent readers and writers to traverse the tree simultaneously without corruption.

The project is purely systems-oriented with a command-line interface. The primary deliverables are the working implementation and a set of live benchmarks demonstrating cache hit rates, throughput scaling under concurrency, and the effect of eviction policy choice, all of which can be run and observed in real time during the evaluation.

2. Weekly Work Plan

2.1. Week 1: Buffer Pool Manager

Goal: Build a fully working buffer pool manager that the B⁺ Tree will sit on top of.

- **Disk Manager:** Implement a flat binary file as simulated disk. Each page is a fixed-size block (e.g., 4 KB). Implement `ReadPage(page_id)` and `WritePage(page_id, data)` to read/write raw bytes from/to this file.
- **Page Table and Frame Array:** Maintain an in-memory array of frames (each frame holds one page worth of bytes). A hash map tracks which `page_id` currently lives in which frame. Each frame has metadata: `pin_count`, `is_dirty`, and access history for LRU-K.
- **LRU-K Replacer:** Implement the LRU-K eviction policy. Each frame records timestamps of its last K accesses. When eviction is needed, the frame with the largest backward k -distance (i.e., oldest K -th access) is chosen as victim. Unpinned frames with fewer than K historical accesses are treated as having infinite distance and are evicted first.
- **Buffer Pool API:** Expose `FetchPage(page_id)`, `NewPage()`, `UnpinPage(page_id, is_dirty)`, and `FlushPage(page_id)`. Pinned pages cannot be evicted. On unpin, dirty pages are written back to disk only when evicted (write-back cache semantics).
- **Unit Tests:** Test with a pool of 5 frames and 20 logical pages. Manually verify eviction order under LRU-1 (standard LRU) and LRU-2 to confirm correctness before moving to the tree.

2.2. Week 2: B⁺ Tree Index

Goal: Build a correct B⁺ Tree where every node is a buffer pool page.

- **Page Layout:** Define the binary layout of internal and leaf pages within a 4 KB page. Internal pages store n keys and $n+1$ child page IDs. Leaf pages store key-value pairs and a pointer to the next leaf (for range scans). Use a header region to store node type, current size, max size, and parent page ID.
- **Search:** Implement point search by traversing from root to the appropriate leaf. At each internal node, binary search to find the correct child pointer, then fetch that child page through the buffer pool.
- **Insertion with Splits:** Insert a key-value pair into the correct leaf. If the leaf overflows (size exceeds max), split it: create a new leaf, redistribute keys, and push the middle key up to the parent. If the parent also overflows, split recursively. If the root splits, create a new root page.
- **Deletion with Merges/Redistribution:** Delete from the leaf. If the leaf underflows (below half capacity), first try to borrow a key from a sibling (redistribution). If neither sibling can spare a key, merge with a sibling and delete the separator key from the parent, recursing upward as needed.

- **Range Scan:** Traverse to the leftmost matching leaf, then follow the leaf-level linked list (next-page pointers) to collect all keys in the range. This tests sequential page access patterns in the buffer pool.
- **Correctness Testing:** Insert 10,000 random integers, verify in-order traversal is sorted. Delete half of them, verify the remaining keys are still retrievable. Use assertions to check tree invariants (node sizes, key ordering) after every operation in debug mode.

2.3. Week 3: Latch Crabbing and Concurrency

Goal: Make the B⁺ Tree safe for concurrent multi-threaded access.

- **Page Latches:** Add a `std::shared_mutex` to each frame in the buffer pool. Reads acquire a shared (read) latch; writes acquire an exclusive (write) latch.
- **Crabbing Protocol for Reads:** When searching, acquire a read latch on the child before releasing the parent's read latch. This allows multiple concurrent readers to traverse different paths simultaneously.
- **Crabbing Protocol for Writes:** When inserting or deleting, acquire write latches top-down. A node is considered *safe* if it will not split or merge (i.e., it has room to absorb a key, or has more than half capacity after a deletion). Once a safe node is identified, release all ancestor write latches, only the path from this safe node to the leaf needs to be held.
- **Multi-threaded Stress Test:** Spawn 8 threads, each performing 50,000 mixed insert/search operations on the same tree. Verify final state with a single-threaded scan for consistency. Use `valgrind -tool=helgrind` or ThreadSanitizer to confirm absence of data races.

2.4. Week 4: Benchmarking, Plots, and Report

Goal: Produce clean, reproducible benchmark results that can be demonstrated live.

- **Benchmark 1: Hit Rate vs Buffer Pool Size:** Fix a workload of 500,000 random searches on a tree with 100,000 keys. Vary the buffer pool size from 32 frames to 2048 frames. Plot cache hit rate (y-axis) vs pool size (x-axis). Expected result: hit rate rises sharply then plateaus.
- **Benchmark 2: LRU-1 vs LRU-K Eviction:** Use a workload with a skewed (Zipfian) access distribution where a small number of pages are accessed very frequently. Compare hit rates of LRU-1 vs LRU-2 and LRU-3. Expected result: LRU-K outperforms LRU-1 on skewed workloads due to protection against one-time sequential scans polluting the cache.
- **Benchmark 3: Throughput vs Thread Count:** Run a mixed workload (70% searches, 30% inserts) with 1, 2, 4, and 8 threads. Measure total operations per second. Expected result: throughput scales with thread count for reads (shared latches allow parallelism) but grows more slowly for writes (exclusive latches serialize).

- **Benchmark 4: Sequential vs Random Insert Patterns:** Insert 200,000 keys in sorted order vs random order. Measure total page I/Os (fetches + writes). Expected result: random inserts cause more page thrashing and higher I/O due to non-sequential access patterns in the buffer pool.
- **Report and Plots:** Generate plots using Python (matplotlib) from benchmark CSV outputs. Write the 10-page final report covering design decisions, implementation challenges, and benchmark analysis.